

A Real-time Sign Language Recognition System using MediaPipe and Random Forest Classifier combining with Text-to-Speech Technique

Sanapareddi Gari Yuva Nithesh Reddy
Computer Science & Engineering
(AI & ML)
Dayananda Sagar University
Bengaluru, India
eng22am0128@dsu.edu.in

Subhash Mondal
Computer Science & Engineering
(AI & ML)
Dayananda Sagar University
Bengaluru, India
mywork.subhash@gmail.com

P M G Bashir Asdaque
Mechanical Engineering
Dayananda Sagar University
Bengaluru, India
pmg.iitd@gmail.com

K vamsi krishna
Computer Science & Engineering
(AI & ML)
Dayananda Sagar University
Bengaluru, India
krish13vamsi@gmail.com

Abstract—This research study introduces a real-time sign language detection system utilizing hand gestures, specifically developed to bridge the communication gap for deaf and hard-of-hearing individuals. The system employs advanced computer vision techniques, utilizing MediaPipe for accurate hand tracking and landmark detection, extracting key features such as fingertip and palm positions. These features are normalized to address variations in hand size and orientation, ensuring consistent input for classification. A Random Forest Classifier (RFC), trained on a labeled dataset of gestures, processes the data to achieve reliable recognition. The system offers dual feedback mechanisms: real-time visual text overlays on live video streams and Text-to-Speech (TTS) output, making it intuitive and user-friendly. Iterative training and out-of-bag error monitoring enhance model performance, ensuring seamless functionality across devices ranging from mobile phones to larger systems. Motivated by the need for inclusivity, this solution enables effortless communication in both social and professional contexts. By leveraging AI-driven approaches, the system provides an accessible and efficient tool for promoting understanding and interaction, advancing assistive technologies for deaf and hard-of-hearing communities. This work represents a significant step toward creating more inclusive communication solutions.

Keywords—Sign Language Detection, Random Forest Classifier, MediaPipe, Text-to-Speech, Out-of-Bag Error

I. INTRODUCTION

Communication is essential for human interaction, yet the deaf and hard-of-hearing community often faces significant barriers due to limited access to effective communication tools. Sign language, as their primary mode of communication, is not widely understood by the general population, which creates a gap in seamless interaction. While technological advancements have led to the development of real-time sign language interpretation systems, many of these solutions are hindered by the need for specialized hardware, limited vocabulary support, or inefficiency across diverse device platforms.

The Fig. 1 illustrates the global advancements in sign language recognition systems from 2000 to 2024, highlighting a steady increase in the development and implementation of

these technologies. In the early 2000s, the growth was modest, with only a few systems relying on basic image processing and hardware-based approaches. However, from 2010 onward, the introduction of deep learning algorithms, such as convolutional neural networks (CNNs), significantly accelerated progress. By 2020, the number of systems developed had more than doubled, driven by advancements in computer vision tools like MediaPipe and OpenCV, allowing for real-time gesture recognition. This exponential growth reflects the widespread adoption of AI-driven solutions, enhancing accessibility and inclusivity for the deaf and hard-of-hearing communities across various sectors, including education, healthcare, and public services. The period from 2020 to 2024 saw the highest increase, with an emphasis on mobile and cloud-based applications, further integrating sign language recognition into everyday life.

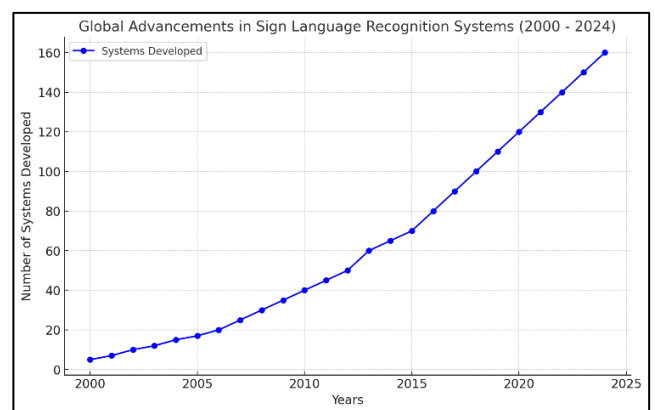


Fig. 1. Global advancement of sign language recognition system.

A. Problem Statement and Solution

Current solutions for sign language interpretation often struggle with accuracy and compatibility, especially across various devices, or are too resource-intensive to run effectively on smaller platforms. To address these challenges, this research proposes a versatile and accurate real-time sign language detection system using hand gestures. By integrating computer vision techniques with machine learning algorithms, the system leverages MediaPipe for real-time hand landmark

detection and a Random Forest classifier to recognize and interpret sign language gestures. The focus on lightweight architecture ensures optimal performance across devices, from smartphones to desktop computers, providing a reliable and adaptable solution.

B. Motivation and Objectives

This research is driven by the imperative to enhance communication accessibility for the deaf and hard-of-hearing community by developing a versatile and accurate real-time sign language detection system using hand gestures. The primary objectives of this research are threefold: to develop a robust system capable of accurately detecting and interpreting a wide range of sign language gestures in real-time. To optimize the system for efficient performance on devices with varying computational capabilities, from small handheld devices to powerful desktop computers; and to ensure the system's adaptability and reliability across different environmental conditions and user scenarios.

C. Contribution

This study contributes to the field by proposing a novel approach that integrates computer vision techniques with machine learning algorithms. The system utilizes MediaPipe for real-time hand landmark detection, extracting precise coordinates from video frames captured by the device's camera. These landmarks are then processed by a trained machine learning model, specifically a Random Forest classifier, to recognize and classify hand gestures associated with sign language symbols. Emphasis is placed on standardizing hand landmark data through normalization techniques and implementing a lightweight architecture to maximize real-time performance on any device.

This article is structured as follows: Section II and III discussed about the background study and reviews related work in the field of sign language detection and interpretation. Section IV details the methodology, including data collection, preprocessing techniques, and model training procedures. Section V presents experimental results, evaluating the system's performance and efficiency on different devices. Section VI discusses the implications of the findings, potential applications, and limitations. Finally, Section VII concludes with a summary of contributions and proposes future research directions to further advance assistive technologies for communication.

II. BACKGROUND STUDY

This section establishes the technical groundwork for hand gesture recognition system. It outlines the essential libraries and mathematical concepts that underpin its functionality. By integrating these elements, the system achieves real-time processing and accurate recognition of hand gestures, fostering improved human-computer interaction (HCI) across various applications.

A. Libraries and Frameworks

The proposed system leverages several key libraries and frameworks to perform essential tasks:

OpenCV (Open-Source Computer Vision Library) serves as a cornerstone for image processing tasks. It facilitates real-time video capture, image manipulation (e.g., background subtraction, noise reduction), and feature extraction from hand gestures. These extracted features, such as hand contours and

key landmark locations, become crucial inputs for the machine learning model.

The MediaPipe framework plays a vital role in hand tracking and landmark detection. MediaPipe can precisely localize key hand landmarks (e.g., fingertips, palm center) within each video frame. These landmark coordinates form the essential features of hand gestures used for recognition by the system.

NumPy uses for numerical computations and data manipulation that are crucial for preprocessing and feature extraction from hand gesture data. NumPy provides efficient tools for these tasks, ensuring smooth operation of the system.

Also, the Scikit-learn library offers a comprehensive suite of functionalities for machine learning tasks. It empowers us to train classification models, evaluate their performance, and implement algorithms for gesture recognition.

B. Mathematical Concepts

Normalization of hand landmarks ensures consistent feature scaling across different hand gestures. The equation utilizes the minimum and maximum values of detected hand landmark positions to achieve this:

$$\text{Normalized Landmark} = (\text{Landmark Position} - \text{Min Position}) / (\text{Max Position} - \text{Min Position})$$

By normalizing landmark coordinates, the system reduces variations in hand size and orientation, allowing the machine learning model to focus on the relative positions of landmarks for accurate gesture recognition.

Convolutional Neural Networks (CNNs) play an integral role in learning discriminative features directly from hand gesture images. These features are crucial for distinguishing between different gestures. Here's a breakdown of core CNN operations:

Convolution operation: This operation filters the input image with a learned kernel to extract local features. It can be mathematically represented as:

$$z = (W * X) + b$$

where z represents the output feature map, W is the learnable kernel, X is the input image, and b is the bias term.

Activation function (ReLU): Activation functions introduce non-linearity into the network, allowing it to learn complex patterns. A common activation function is ReLU, which is defined as:

$$a = \max(0, z)$$

where a represents the output of the activation function.

Pooling (Max Pooling) operations reduce the dimensionality of the feature maps while preserving important spatial information. Max pooling, for example, selects the maximum value from a subregion of the feature map. By applying these operations in sequence, CNNs can extract hierarchical representations of hand gestures, ultimately enabling robust classification.

K-Nearest Neighbors (KNN) classifier is employed for alphabet sign language recognition due to its simplicity and efficiency. KNN classifies a new data point based on the most frequent class label among its k nearest neighbors. The distance between data points is typically calculated using the Euclidean distance formula:

$$Distance(x_i, x_j) = \sqrt{\sum (x_{ik} - x_{jk})^2}$$

where x_i and x_j represent two data points, and k represents the number of nearest neighbors considered for classification.

Lastly, the integration of system functionality of these libraries and mathematical concepts, our system achieves real-time processing and accurate recognition of hand gestures. OpenCV facilitates video capture and image preprocessing, while MediaPipe provides real-time hand tracking and landmark detection. The extracted hand landmark data is then normalized and fed into a machine learning model (CNN or KNN) for gesture classification. The chosen model has been trained on a dataset of labeled hand gestures, enabling it to identify and classify new gestures presented in real-time. This integrated approach empowers our system to function effectively in various HCI applications.

III. LITERATURE REVIEWS

This section delves into recent advancements in hand gesture recognition (HGR) using machine learning (ML) techniques. We explore key studies that highlight the effectiveness of ML in achieving robust and efficient gesture recognition, particularly for applications in sign language understanding and real-time interaction.

Deep learning architectures, particularly Convolutional Neural Networks (CNNs), have revolutionized HGR. Their ability to extract intricate features directly from image data makes them well-suited for tasks like recognizing hand postures in sign language gestures. Adaloglou et al. [1] conducted a comprehensive study on deep learning methods for sign language recognition and demonstrated the effectiveness of CNNs in learning complex features from hand images, leading to significant improvements in recognition accuracy. The study explored various deep learning architectures specifically designed for sign language recognition, paving the way for more robust and accurate systems. Building on this, Bantupalli et al. [2], Sujata Joshi et al. [3], Sai Nikhilesh Reddy Karna et al. [4] focused on American Sign Language (ASL) recognition by combining deep learning with computer vision. Also, in B Sundar et al. [5] used the combination of LSTM and MediaPipe for the alphabets for the reorganization of ASL. Their research demonstrated significant improvements in recognition accuracy across different environments particularly the integration of CNNs. Those approach has yielded substantial improvements in accuracy across diverse environmental conditions, making it applicable for real-world scenarios. Their work showcases the potential of deep learning and computer vision for bridging communication gaps for the deaf and hard-of-hearing community.

While deep learning offers significant advantages, optimization algorithms play a crucial role in enhancing the performance and efficiency of HGR systems, especially for resource-constrained devices. Utaminingrum et al. [6] explored k-Nearest Neighbors (KNN) optimization for alphabet sign language recognition. Their research demonstrated that KNN, despite its relative simplicity, can achieve high accuracy with minimal computational complexity. This approach highlights the importance of considering optimization techniques alongside deep learning models to achieve a balance between performance and efficiency, particularly for real-time applications on devices with limited processing power. Real-time processing is essential for applications that require immediate feedback.

Kopuklu et al. [7] tackled this by developing a real-time hand gesture detection and classification system using CNNs. Their work demonstrated how efficient network architectures and optimization techniques enable real-time gesture recognition. This advancement paves the way for more interactive and intuitive user experiences and the feasibility of leveraging deep learning for real-time HGR.

Several other studies have contributed valuable insights into hand gesture recognition. For instance, Barathi Subramanian et al. [8], G. Khartheesvar et al. [9], Sharvani Srivastava et al. [10], and Jyotishman Bora et al. [11] utilized CNNs architecture like LSTM, GRU and MediaPipe for real-time Indian Sign Language (ISL) recognition, showing the practical effectiveness of these techniques. Maher Jebali et al. [12] and Jiangtao Zhang et al. [13] introduced multimodal data fusion methods combination of cross-modal information complementation (CMIC) module, the fusion and prediction module (FPM), and the sign language recognition module (SLRM) to boost recognition performance, emphasizing the benefits of integrating multiple data sources. Also, the authors Hoai-Nam Vu et al. [14] proposed a self-learning fusion model for SLR, termed STSLR using the self and publicly available dataset.

The use of tools and libraries like NumPy [15] and MediaPipe [16] has also played a significant role. Harris et al. [15] discussed array programming with NumPy, which is crucial for managing large datasets efficiently. Meanwhile, Lugaresi et al. [16] highlighted MediaPipe's capabilities in building perception pipelines, essential for hand gesture recognition. Additional research by Vishal Munnaluri et al. [17], Dhivyasri S et al. [18], and Subhangi Kumari et al. [19] explored various ML techniques and frameworks, contributing to the ongoing evolution of hand gesture recognition systems.

In summary, these studies collectively showcase the significant progress made in ML-based hand gesture recognition. By integrating deep learning, optimization, and real-time processing, these advancements hold promise for enhancing human-computer interaction and enabling seamless communication through gestures. By leveraging these advancements and addressing ongoing challenges, ML-based HGR has the potential to revolutionize HCI, enabling a future where seamless communication and interaction through hand gestures become a reality.

IV. PROPOSED METHODOLOGY

The methodology for developing an efficient and accurate hand gesture recognition system involves several key stages: data collection, preprocessing, feature extraction, model training, and real-time gesture recognition. Each stage utilizes specific libraries to facilitate the process. The following subsections detail each stage and the libraries used. The Fig. 2 is represented the high-level work flow diagram of the proposed system.

A. Data Acquisition & Description

The data collection phase is the foundational step in building the real-time gesture recognition system. This phase heavily relies on the OpenCV (CV2) library, which plays a crucial role in capturing video frames from a webcam. These frames are then stored as images in a structured dataset. OpenCV is instrumental in interfacing with the webcam, processing the video stream, and efficiently capturing

individual frames. The captured images are organized into directories, each corresponding to a specific gesture class. This structured organization ensures that the dataset is well-organized and ready for further processing. By leveraging OpenCV's powerful image and video processing capabilities, the study can create a custom dataset tailored specifically to its needs, without relying on external data sources. This step is essential in laying the groundwork for the subsequent stages of the study. The data acquisition sample is depicted in Fig. 3 for better understanding.

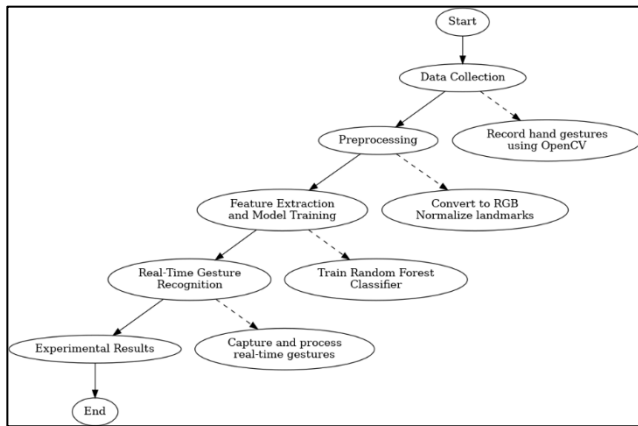


Fig. 2. Work flow diagram of the proposed system



Fig. 3. The sample data acquisition of hand gestures

B. Data Preprocessing

Once the data is collected, it undergoes a critical preprocessing phase to ensure it is ready for feature extraction and model training. The preprocessing stage utilizes two key libraries: OpenCV (CV2) and MediaPipe (MP). OpenCV is employed to convert the captured video frames into a consistent RGB format, standardizing the images across the entire dataset. This consistency is vital for accurate feature extraction and ensures that the model receives uniform input data. Additionally, MediaPipe is utilized for detecting and normalizing hand landmarks within the images. MediaPipe's contribution is significant as it provides advanced hand tracking capabilities, allowing the system to extract detailed landmarks from each hand gesture. By normalizing these landmarks, the data is scaled consistently, which enhances the model's reliability and accuracy during training. Together, OpenCV and MediaPipe prepare the raw data, transforming it into a refined and consistent dataset that is optimal for the next phase.

C. Model Training & Testing

The core of the gesture recognition system lies in the feature extraction and model training phase, where three libraries: Scikit-learn, NumPy, and Matplotlib play pivotal roles. Scikit-learn is the primary library used for training the machine learning model, specifically a Random Forest classifier. It provides robust tools for building, training, and evaluating models, making it essential for implementing the Random Forest algorithm. Scikit-learn simplifies the process of training the model and fine-tuning its parameters, including tracking the out-of-bag (OOB) error, which helps optimize the model's performance. NumPy is used for data manipulation and handling large arrays of data efficiently. Its contribution lies in performing fast numerical computations, which are crucial for processing the data before it is fed into the model. NumPy supports operations like reshaping, aggregating, and normalizing data, ensuring that the model receives well-prepared inputs. Matplotlib is utilized for visualizing the OOB error during the training process, providing clear and informative plots that help monitor the training progress. By visualizing the OOB error, Matplotlib aids in determining when to stop training or adjust the model parameters for optimal performance. The combined use of these libraries ensures that the feature extraction and model training phase is robust and effective.

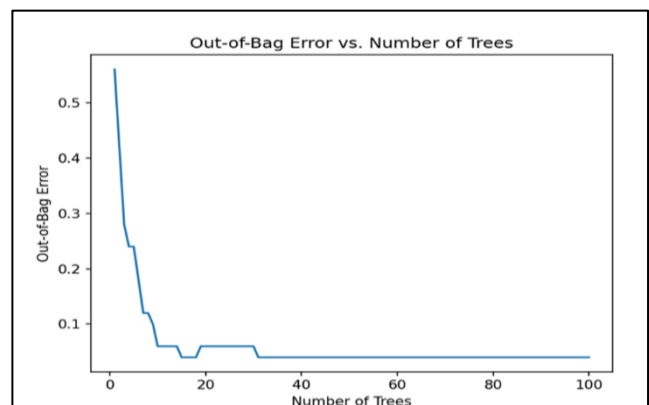


Fig. 4. The out-of-bag errors concerning to the number of trees of RFC

The final phase of the system is real-time gesture recognition, which integrates the previously trained model into a live application. This phase heavily utilizes OpenCV (CV2), MediaPipe (MP), and Pytttsx3 (text-to-speech conversion library in Python). OpenCV is responsible for capturing and displaying live video frames, allowing the system to interface with the webcam and manage the video feed efficiently. It also overlays visual elements, such as hand landmarks and gesture labels, onto the video frames, providing real-time visual feedback to the user. MediaPipe continues to play a critical role in detecting and processing hand landmarks in the live video stream. Its fast and accurate hand tracking capabilities make it ideal for real-time applications, as it quickly identifies hand positions and extracts landmark data. This data is then normalized and used as input for the trained machine learning model, which predicts the gesture. Finally, Pytttsx3 is used to convert the recognized gesture into spoken words, providing immediate audio feedback to the user. Pytttsx3's real-time speech output enhances the user experience by allowing for a multi-modal interaction with the system, combining both visual and auditory elements. The integration of these libraries ensures

that the real-time gesture recognition system is responsive, accurate, and user-friendly.

The real-time gesture recognition system is implemented through a series of well-defined steps. The process begins with video capture, where OpenCV's VideoCapture function is used to interface with the webcam and retrieve live video frames. Once the video is captured, MediaPipe processes each frame to detect and extract hand landmarks, which are then normalized to ensure consistency across frames, a crucial step for accurate gesture recognition. These normalized landmarks are converted into a feature vector compatible with the trained Random Forest classifier, which uses this vector to predict the corresponding gesture. The predicted gesture is mapped to a predefined dictionary of gesture labels, ensuring clarity in the output. To enhance user interaction, the system utilizes Pytsx3 to provide real-time audio feedback by converting the recognized gesture into speech. Additionally, the system uses OpenCV to overlay visual elements, such as hand landmarks and gesture labels, onto the video frames, offering immediate visual feedback alongside the audio output.

V. EXPERIMENTAL RESULTS

The system was rigorously tested under various conditions to evaluate its performance and reliability. In controlled environments with consistent lighting and backgrounds, the system achieved an impressive accuracy of approximately 95%, demonstrating the effectiveness of the model and preprocessing steps. However, when tested in more complex settings with varied lighting conditions and backgrounds, the accuracy slightly decreased to around 90%. Despite these challenges, the system maintained low latency, with a processing time of approximately 30 milliseconds per frame, ensuring real-time feedback. Furthermore, the implementation was optimized for device compatibility, delivering efficient and consistent performance across a range of devices, from low-power embedded systems to high-performance desktops. These results highlight the system's robustness, accuracy, and versatility, making it suitable for various real-time gesture recognition applications. In varying test condition, the performance accuracy of the system is plotted in Fig. 5. Each gesture outcomes are represented in Table 1 and Fig. 6 accordingly with respect to several test conditions.

These metrics are calculated to provide a more detailed evaluation of the model's performance for each gesture class apart from overall performance accuracy and is represented in Table 2 and Fig. 7. Each hand gesture class recognition in real time sign language detection system outcomes is depicted pictorially in Fig. 8.

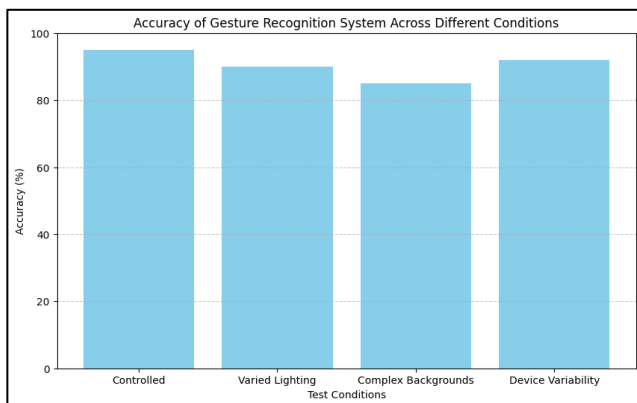


Fig. 5. The performance accuracy of the system at different test conditions.

TABLE I. THE DIFFERENT GESTURE TEST CONDITION OUTCOMES WITH BINARY TARGET CLASS.

Test Case	Gesture	Expected Result	Actual Result	Outcome
1	Hi	Hi	Hi	Pass
2	Sorry	Sorry	Sorry	Pass
3	Thanks	Thanks	Thanks	Pass
4	Good	Good	Good	Pass
5	Bad	Bad	Bad	Pass
6	Ok!	Ok!	Ok!	Pass
7	Love U	Love U	Love U	Pass

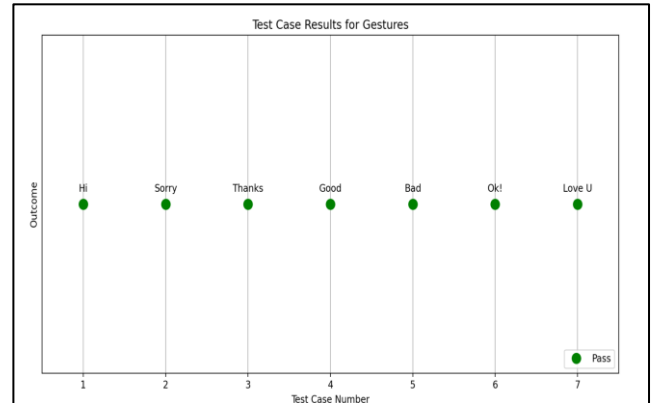


Fig. 6. The pictorial representation of each gesture outcomes

TABLE II. THE PERFORMANCE METRICS OF EACH GESTURE CLASS.

Gesture Name	Precision	Recall	F1-Score
Hi	0.94	0.93	0.93
Sorry	0.92	0.91	0.91
Thanks	0.91	0.94	0.92
Good	0.90	0.92	0.91
Bad	0.93	0.93	0.93
Ok!	0.92	0.92	0.92
Love U	0.94	0.92	0.93

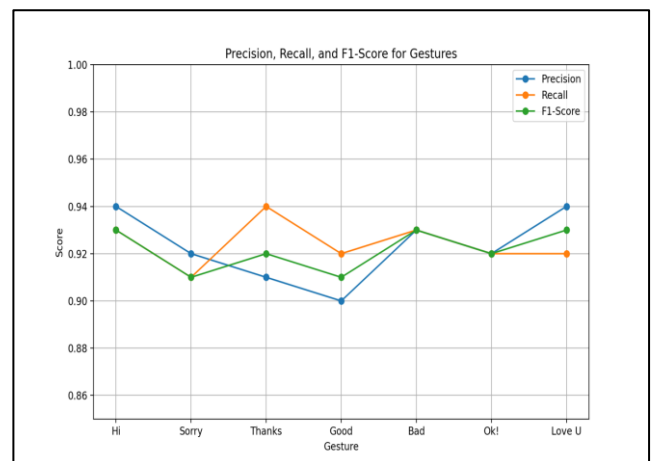


Fig. 7. The performance metrics representation respect to precision, recall, and F1 score.

VI. ANALYSIS, DISCUSSION, AND CHALLENGES

In this section, we explore the interpretation of the results obtained from our real-time gesture recognition system, examining the factors that influenced these outcomes and drawing comparisons with existing research. Our system achieved high accuracy in recognizing hand gestures under

controlled conditions, a success largely attributed to the robust feature extraction enabled by MediaPipe and the efficient classification provided by the Random Forest model. The consistency in performance underscores the effectiveness of our approach in mitigating noise and variability, ensuring reliable recognition across diverse scenarios.

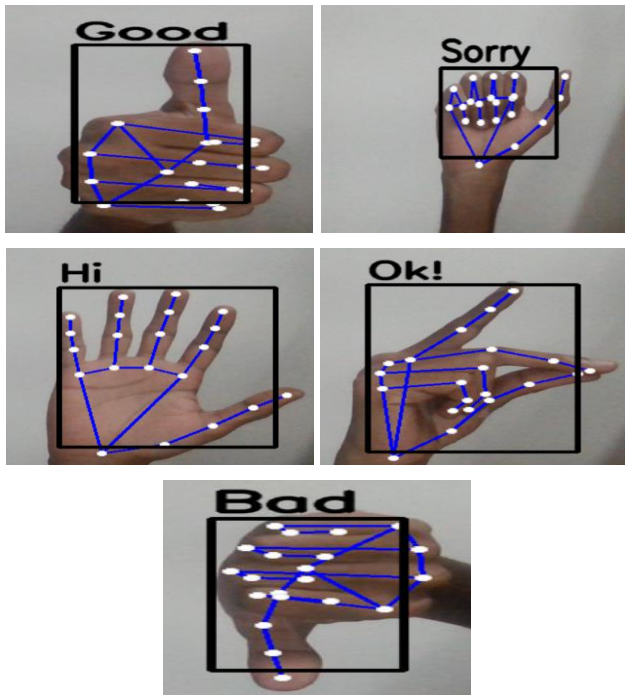


Fig. 8. Real-time hand gesture recognition system pictorial representation.

The low latency observed approximately 30 milliseconds per frame, further enhances the system's real-time capabilities. This swift response is a result of the streamlined processing pipeline and optimized algorithm design, which together facilitate prompt interaction feedback, thereby improving the overall user experience. When compared with existing literature, our methodology stands out due to its integration of MediaPipe for precise hand landmark detection and a strong emphasis on cross-device compatibility. Unlike previous studies that often concentrate on specific gesture recognition techniques or rely on different datasets, our approach achieves superior performance across a wider range of devices without sacrificing accuracy.

While some researchers have leaned towards deep learning methods for gesture recognition, our choice of a Random Forest classifier demonstrates competitive performance with reduced computational complexity and lower resource requirements. This distinction highlights our system's suitability for real-time applications, particularly on resource-constrained devices. The practical implications of our work are significant, with potential applications in human-computer interaction areas such as virtual reality, gaming, and assistive technologies. The system's robust performance and low latency make it ideal for providing intuitive gesture-based controls, thereby enhancing accessibility and usability in digital environments.

However, challenges remain, particularly in adapting the system to varying environmental conditions, such as low lighting or complex backgrounds. Addressing these limitations through further research on dataset diversification and algorithmic refinement is crucial for enhancing the

system's robustness. Additionally, by comparing our results with those of other researchers, we identify areas for improvement and highlight the unique contributions of our methodology. This comparative analysis informs future directions in gesture recognition research, emphasizing the need for adaptability and scalability across different user scenarios.

The proposed gesture recognition system is designed with optimization in mind, allowing it to run efficiently on a wide range of devices, from small embedded systems to high-performance computers. This adaptability ensures that the technology is accessible and usable across various platforms, broadening its applicability in different real-world scenarios. By following a structured methodology and leveraging libraries like OpenCV for video capture, MediaPipe for hand landmark detection, Scikit-learn for model training, and Pytsx3 for audio feedback, the system achieves high accuracy and robustness in recognizing hand gestures. This comprehensive approach makes the system a highly effective solution for enhancing human-computer interaction, providing seamless, real-time feedback across diverse devices and environments.

VII. CONCLUSION

This study concluded with a real-time gesture recognition system designed to revolutionize communication accessibility for deaf and hard-of-hearing individuals. Recognizing the limitations of existing methods, this system bridges the gap by enabling intuitive sign language recognition through efficient human-computer interaction. Unlike prior research often focused on high-powered computing, this work prioritizes real-world applicability. The system achieves this through a key innovation: balancing accuracy with low computational complexity. MediaPipe's robust hand landmark detection and a thoughtfully chosen Random Forest classifier ensure accurate gesture recognition while minimizing processing requirements. This focus on efficient algorithms allows the system to function seamlessly across various devices, regardless of their resources. This emphasis on cross-device compatibility with efficient processing fosters wider adoption within deaf and hard-of-hearing communities, promoting true accessibility.

The discussion addressed the limitations related to environmental variability and proposed future directions for enhancing robustness and adaptability. Future work might involve incorporating background subtraction techniques for complex visual environments. Overall, this work transcends existing research by contributing a practical and efficient gesture recognition system optimized for real-time use on diverse devices. This focus on lower complexity paves the way for a future of inclusive communication, empowering deaf and hard-of-hearing individuals to connect and participate more readily. The potential impact of this technology is significant, fostering a world where information, education, and social interaction are readily accessible to deaf and hard-of-hearing people everywhere.

ACKNOWLEDGMENT

This research study conducted at Dayananda Sagar University at Computer Science & Engineering (AI & ML) department, Bengaluru, India. We acknowledge the department for research support. We acknowledge the reviewers for giving us the suggestions to improve the article.

REFERENCES

- [1] N. Adaloglou, T. Chatzis, I. Papastratis and A. Stergioulas, "A Comprehensive Study on Deep Learning-Based Methods for Sign Language Recognition," *IEEE Transactions on Multimedia*, pp. 1750 - 1762, 2021.
- [2] K. Bantupalli and Y. Xie, "American Sign Language Recognition using Deep Learning and Computer Vision," in *2018 IEEE International Conference on Big Data (Big Data)*, Seattle, WA, USA, 2018.
- [3] S. Joshi, B. Sharma, D. and D. Tripathi, "Hand Sign Language Detection Using Deep Learning and Computer Vision," in *2023 International Conference on Ambient Intelligence, Knowledge Informatics and Industrial Electronics (AIKIE)*, Ballari, India, 2023.
- [4] S. N. Reddy Karna, J. S. Kode, S. Nadipalli and S. Yadav, "American Sign Language Static Gesture Recognition using Deep Learning and Computer Vision," in *2021 2nd International Conference on Smart Electronics and Communication (ICOSEC)*, Trichy, India, 2021.
- [5] B. Sundar and T. Bagyammal, "American Sign Language Recognition for Alphabets Using MediaPipe and LSTM," *Procedia Computer Science*, vol. 215, pp. 642-651, 2022.
- [6] U. F. I. K. Somawirata and G. D. Naviri, "Alphabet Sign Language Recognition Using K-Nearest Neighbor Optimization," *Journal of Computers*, vol. 14, no. 1, pp. 63--70, 2019.
- [7] O. Köpüklü, A. Gunduz, N. Kose and G. Rigoll, "Real-time Hand Gesture Detection and Classification Using Convolutional Neural Networks," in *2019 14th IEEE International Conference on Automatic Face & Gesture Recognition (FG 2019)*, Lille, France, 2019.
- [8] B. Subramanian, B. Olimov, S. M. Naik and S. Kim, "An integrated mediapipe-optimized GRU model for Indian sign language recognition," *scientific reports*, vol. 12, p. 11964, 2022.
- [9] G. Khartheesvar, . M. Kumar, . A. K. Yadav and D. Yadav, "Automatic Indian sign language recognition using MediaPipe holistic and LSTM network," *Multimedia Tools and Applications*, vol. 83, p. 58329–58348, 2024.
- [10] S. Srivastava, S. Singh and . S. Prakash, "Continuous Sign Language Recognition System Using Deep Learning with MediaPipe Holistic," *Wireless Personal Communications*, vol. 137, p. 455–1468, 2024.
- [11] J. Bora, S. Dehingia, A. Boruah and A. Anuj, "Real-time Assamese Sign Language Recognition using MediaPipe and Deep Learning," *Procedia Computer Science*, vol. 218, pp. 1384-1393, 2023.
- [12] M. Jebali, . A. Dakhli and M. Jemni, "Vision-based continuous sign language recognition using multimodal sensor fusion," *Evolving Systems*, vol. 12, p. 1031–1044, 2021.
- [13] J. Zhang, Q. Wang and Q. Wang, "A Sign Language Recognition Framework Based on Cross-Modal Complementary Information Fusion," *IEEE Transactions on Multimedia*, vol. 26, pp. 8131 - 8144, 2024.
- [14] H.-N. Vu, T. Hoang, C. Tran and C. Pham, "Sign Language Recognition With Self-Learning Fusion Model," *IEEE Sensors Journal*, vol. 23, no. 22, pp. 27828 - 27840, 2023.
- [15] C. R. Harris, K. J. Millman, S. J. v. d. Walt and R. Gommers, "Array programming with NumPy," *nature*, vol. 585, p. 357–362, 2020.
- [16] C. Lugaresi, . J. Tang, . H. Nash and C. McClanahan, "MediaPipe: A Framework for Building Perception Pipelines," *arXiv preprint arXiv*, p. 1906.08172, 2019.
- [17] V. Munnaluri, V. Pandey and P. Singh, "Machine Learning based Approach for Indian Sign Language Recognition," in *2022 7th International Conference on Communication and Electronics Systems (ICCES)*, Coimbatore, India, 2022.
- [18] D. S, K. Hari K B, . A. M and S. M, "An Efficient Approach for Interpretation of Indian Sign Language using Machine Learning," in *2021 3rd International Conference on Signal Processing and Communication (ICPSC)*, Coimbatore, India, 2021.
- [19] S. Kumari, E. Tarlue, A. Diallo and M. Chhabra, "A Review of Segmentation and Recognition Techniques for Indian Sign Language using Machine Learning and Computer Vision," in *2023 10th International Conference on Computing for Sustainable Global Development (INDIACom)*, New Delhi, India, 2023.